

Helix Thready — System Overview (C4 Context & Container)

Table of Contents

1. Purpose & scope

2. What Helix Thready is

3. C4 Level 1 — System context

4. C4 Level 2 — Container view

5. Architectural drivers (scale, SLO, tenancy)

6. Cross-cutting concerns

7. Verified vs assumed

8. Gap-register coverage

9. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (System Architecture)	Initial draft — C4 context+container, scale, SLOs, cross-cutting
2	2026-07-22	swarm (Pass 3 depth)	Split both C4 diagram explanations (context §3, container §4) into true multi-paragraph form per CONVENTIONS §4 — trust-boundary/actor-RBAC/messenger-duplex/outbound-only for context; five-plane failure-domain reading + the two load-bearing loop edges for container

Table of Contents

1. Purpose & scope

2. What Helix Thready is

3. C4 Level 1 — System context

4. C4 Level 2 — Container view

5. Architectural drivers (scale, SLO, tenancy)

6. Cross-cutting concerns
7. Verified vs assumed
8. Gap-register coverage
9. Open items

1. Purpose & scope

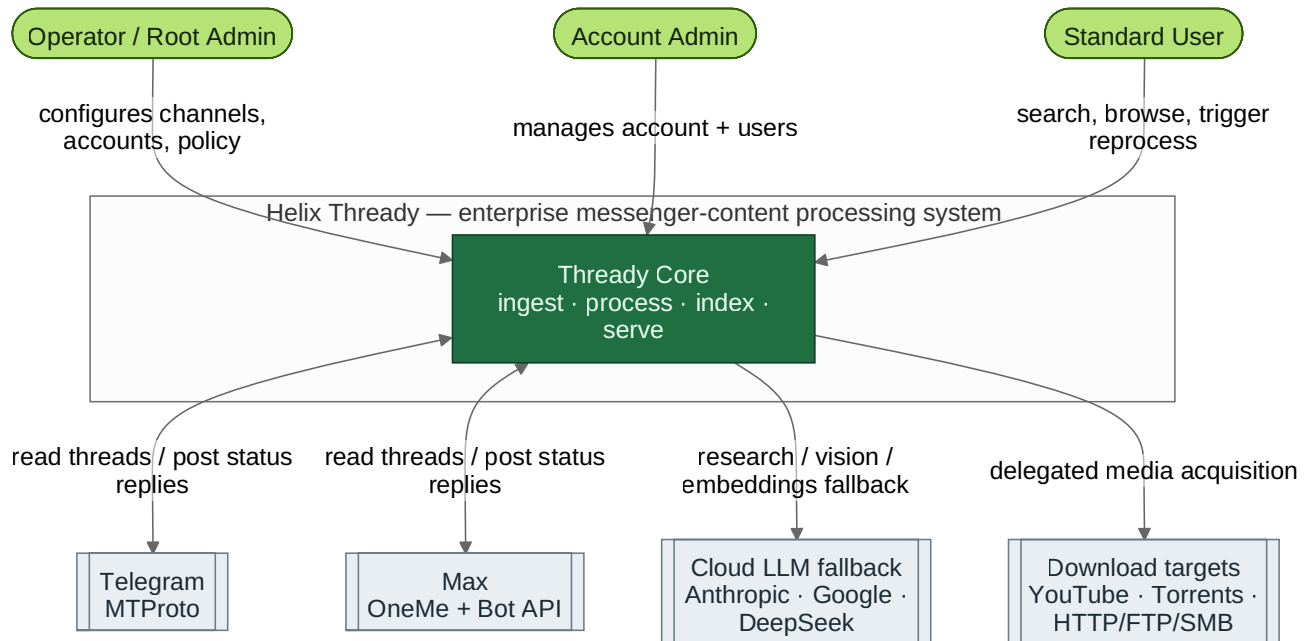
This document is the top-level architectural map of **Helix Thready** (`helix_thready`). It gives the C4 context and container views and states the drivers (scale, SLO, tenancy) that shape every downstream decision. It never re-decides technology — the canonical decision matrix lives in `helix_thready_research_request_final.md §0.2` and is treated as read-only. Detailed subsystem designs are delegated to the sibling files linked in the header.

2. What Helix Thready is

Helix Thready connects to messaging platforms — **initially Telegram and Max** — using the operator’s private accounts, reads the **complete thread** (root post + the full chain of organic replies), persists it to a relational store and a semantic (vector) store, classifies each post by hashtag + content type, dispatches AI-powered **Skills** (recipes) to process it (download, convert, analyze, research), posts a **status reply** back to the source post, and exposes everything through a REST `/v1` API and a real-time event bus. It is event-driven, idempotent (no double-processing of the same post), multi-tenant (three-tier RBAC), and white-labelable. `[research_request_final §1.2–1.3]`

The single most load-bearing functional requirement, called out explicitly in the original request, is **thread assembly**: hashtags are frequently added as a *reply* to a link-only or text-only root post, so a “post” is never a single message — it is the root plus its organic reply chain, excluding the system’s own status replies. This drives the ingestion design (`messenger-ingestion.md`) and the idempotency model (`concurrency-and-idempotency.md`).

3. C4 Level 1 — System context



Rendered PNG/SVG exported via Docs Chain (\$11.4.65). Source: [diagrams/c4-context.mmd](#).

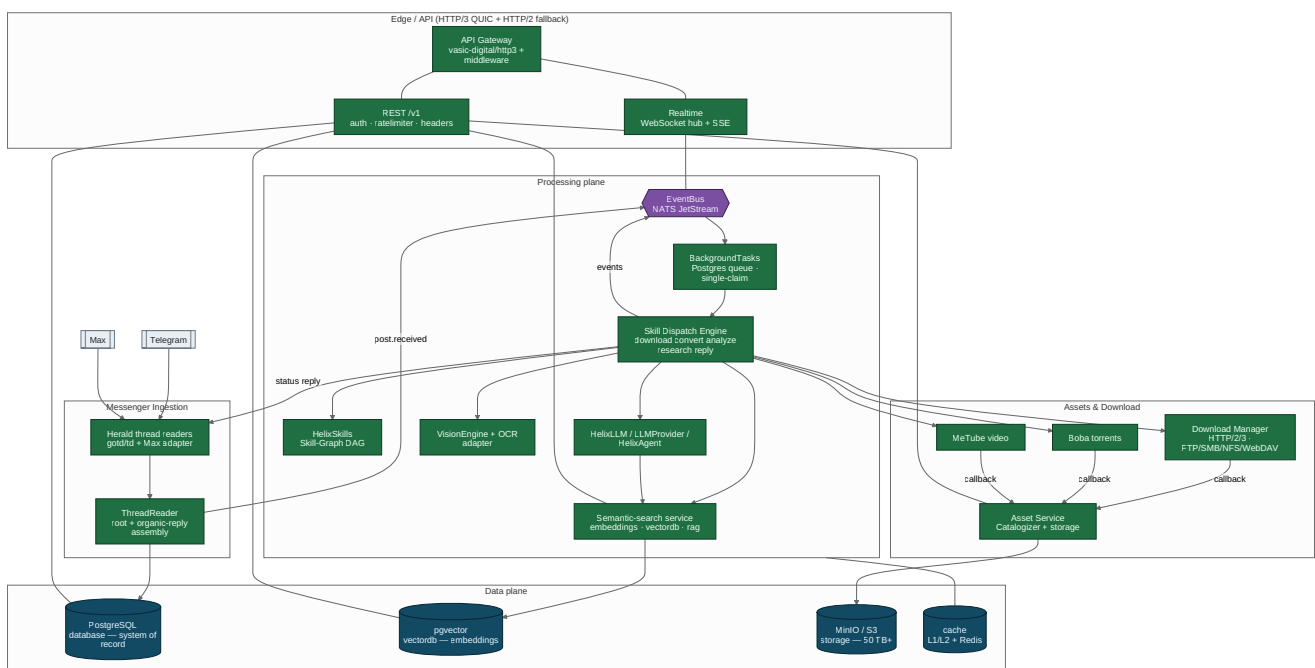
Explanation (for readers/models that cannot see the diagram). The context diagram exists to fix the *trust boundary* of the whole system before any internal structure is drawn: it places one box — *Thready Core* — at the centre and shows only the actors and external systems that cross that boundary. Reading it is reading the answer to “who talks to Thready, and in which direction?”, which is what every later security, tenancy and integration decision is derived from. Nothing a client can see bypasses the centre box; that single fact is the root of the mediation invariant stated at the end of this explanation.

Three human actor roles sit on the left, and they are drawn as three *distinct* roles precisely because they mirror the three-tier RBAC hierarchy one-for-one. The **Operator / Root Admin** — of which exactly one exists — configures the monitored channels, the messenger accounts and the global policy; the **Account Admin** manages their own account and its users and sets that account’s branding; the **Standard User** only consumes — searching, browsing generated materials, and triggering reprocessing. Modelling the actors as the RBAC tiers here (rather than as a single undifferentiated “user”) is deliberate: it makes visible at the very top level that authority is scoped *per account*, which [security-model.md](#) then enforces on every request.

On the external-system side there are two very different kinds of boundary, and the diagram distinguishes them by arrow direction. **Telegram** and **Max** are *bidirectional*: Thready reads thread history *from* them — using user-level protocols (MTPROTO for Telegram, OneMe + Bot API for Max), not merely bot APIs, because bots cannot backfill arbitrary channel history — and posts *status replies* back *to* them. That read-and-reply duplex is the reason ingestion and the reply step are the two ends of one messenger relationship rather than two separate integrations ([messenger-ingestion.md](#)).

The other two external boundaries are strictly *outbound* and only ever reached indirectly. The **Cloud LLM fallback** (Anthropic, Google, DeepSeek via `LLMProvider`) is used solely when the local HelixLLM stack cannot serve a request, so it is a degradation path, not a primary dependency. The **Download targets** boundary (YouTube, torrents, and HTTP/FTP/SMB/NFS/WebDAV sources) is always reached through the delegated download layer, never by clients directly. The payoff of drawing these as outbound-only is the mediation invariant: everything a client can see is produced by Thready Core, so clients never receive direct file URLs, raw messenger tokens, or a cloud provider's response unmediated — the boundary the diagram draws is also the boundary every credential and asset respects.

4. C4 Level 2 — Container view



Rendered PNG/SVG exported via Docs Chain (\$11.4.65). Source: `diagrams/c4-container.mmd`.

Explanation (for readers/models that cannot see the diagram). The container diagram decomposes Thready into five planes, and the grouping is not cosmetic: each plane is a *failure- and scaling-domain* boundary, so a plane can be reasoned about, deployed and scaled largely independently of the others. Read top to bottom it follows a request from the network edge down to durable storage, but the more important reading is horizontal — which plane owns which concern — because that ownership is what keeps the aggressive API SLO isolated from a 30-minute Skill run.

The **Edge / API** plane terminates client traffic over HTTP/3 (QUIC) with HTTP/2 fallback via `vasic-digital/http3`, applies auth (`digital.vasic.auth`), rate limiting (`digital.vasic.ratelimiter`) and security headers (`security/pkg/headers`), and exposes a versioned REST `/v1` surface plus a realtime surface (a WebSocket hub + Server-Sent-Events for one-way streams). Immediately behind it, the **Messenger Ingestion** plane is the extended Herald: platform readers (Telegram via `gotd/td`, Max via a new adapter)

feed a shared **ThreadReader** that assembles the root + organic reply chain, writes raw posts to PostgreSQL, and emits a `post.received` event. These two planes are the only ones that touch the outside world (clients on one side, messengers on the other), which is why every authentication and thread-assembly concern is concentrated there rather than smeared across the system.

The **Processing plane** is the heart of the system and the reason the whole architecture is event-driven. The `post.received` event lands on the **EventBus** (NATS JetStream), is claimed *exactly once* by the Postgres-backed **BackgroundTasks** queue, and is handed to the **Skill Dispatch Engine**, which orders and runs the matching Skills (resolved from the HelixSkills Skill-Graph) using the LLM stack (HelixLLM / LLMPProvider / HelixAgent), VisionEngine + OCR, the download layer, and the semantic-search service. Because this plane sits *behind* the EventBus + queue rather than on the request thread, a Skill can run for half an hour without ever threatening the < 150 ms API budget — the decoupling visible here is the concrete mechanism behind the latency-isolation driver of \$5.

The **Assets & Download** plane fetches bytes (Download Manager for direct protocols, Boba for torrents, MeTube for streaming video) and hands completed artifacts to the **Asset Service** (built on Catalogizer), which stores them in the MinIO/S3 object tier. The **Data plane** underneath holds the system of record (PostgreSQL), the *co-located* embedding index (pgvector), the object store (MinIO/S3 for the 50 TB+ target), and the L1/L2 cache. Keeping the four datastores in one plane makes the system-of-record boundary explicit: the relational store is authoritative and everything else — vectors, cache, even the durable event log — is derived and rebuildable from it (`data-flow.md` \$4).

Two edges are worth stressing because they are easy to miss and load-bearing. First, the processing plane both *consumes from* and *produces to* the EventBus — the arrow back to the bus (and onward to Herald as the status reply) closes the event loop, so the pipeline's own outputs re-enter the same durable transport that fed it, which is what lets a reprocess or a status reply be just another event rather than a special case. Second, semantic indexing (`SEM → PGV`) happens for both original posts and every generated artifact, which is what makes “search by meaning” work across the whole corpus.

5. Architectural drivers (scale, SLO, tenancy)

These operator decisions (`$0.1`) are non-negotiable inputs to every subsystem:

Driver	Value	Architectural consequence	Provenance
Scale	Large / multi-tenant: 100+ channels, 10k+ posts/day, 100+ users, 50 TB+ assets	Postgres partitioning + read replicas; JetStream as primary transport; MinIO/S3 object tier; horizontal scaling from day one	[OPERATOR]
SLO	Aggressive: API p95 < 150 ms, semantic search < 500 ms, page < 1.5 s	Async processing with progress events (never block the API on a Skill); pgvector ANN index tuning; cache tiers	[OPERATOR]
Tenancy	Three-tier RBAC (Root / Account Admin / User); white-labeling	Every row carries <code>account_id</code> ; policy enforcement at the API and Asset Service; per-account branding	[OPERATOR] [research_request_fi nal \$6]
Retention	Keep indefinitely, per-account overrides	Time-partitioned posts; retention/archive helpers; per-account cleanup policy	[OPERATOR]
Backup/DR	Daily full + hourly DB incrementals; RPO ≈ 1 h, RTO ≈ 4 h	PITR from incrementals; asset snapshot/dedup; documented restore runbook	[OPERATOR]
Compliance	Internal/private — minimal	Encryption + secrets hygiene mandatory; GDPR-aware hooks (erasure/export) but no formal certification for MVP	[OPERATOR]

The 10k+ posts/day figure (~0.12 posts/s average, with bursts) is comfortably within a single Postgres+JetStream deployment; the design constraint is not raw throughput but **latency isolation** — a research-heavy Skill can run for 30 minutes, so processing must never share a request thread with the API. This is why processing is fully decoupled behind the EventBus + BackgroundTasks queue. See [processing-pipeline.md](#).

6. Cross-cutting concerns

- **In-house first** [CONSTITUTION §11.4.28]. Almost every capability maps to an existing owned Go submodule (`digital.vasic.*` , `HelixDevelopment/*`); only six items are genuinely new ([BUILD-NEW]): Asset Service (decouple Catalogizer), Download Manager, User Service, Max adapter, OCR adapter, MeTube completion webhook — plus the thin Event Bus service, ThreadReader abstraction, and Semantic-search service that wrap existing engines. The full mapping and maturity is in `component-catalog.md`.
- **Event-driven & idempotent.** All state changes emit events; the same post is claimed and processed exactly once. See `event-model.md` and `concurrency-and-idempotency.md`.
- **Security & tenancy.** AES-256-GCM at rest, TLS 1.3 in transit, three-tier RBAC, sensitive content sealed but semantically searchable. See `security-model.md`.
- **Discovery & dynamic ports.** `discovery` + `mdns` + `port_prefix` give deterministic host-port bands and self-registration. See `service-discovery.md`.
- **Observability.** OpenTelemetry + Prometheus + logrus + ClickHouse via `digital.vasic.observability` ; per-container health checks (`observability/pkg/health`).
- **No server-side CI** [CONSTITUTION §11.4.156]. Local git-hooks + pre-tag full-suite retest
 - all-four-upstreams push.

7. Verified vs assumed

- **VERIFIED (read at source):** the container-plane engines exist and expose the interfaces used here — `EventBus` (`pkg/event` , `pkg/bus` , `pkg/nats` JetStream), `BackgroundTasks` (`TaskQueue` / `TaskExecutor` / `TaskRepository` interfaces, Postgres claim), `herald` (`pkg/messenger.Messenger` interface), `discovery` / `port_prefix` , `security/pkg/securestorage` .
- **ASSUMPTION / DESIGN:** the *composition* shown here (ThreadReader, Skill Dispatch Engine, Semantic-search service, Asset Service, Download Manager) is Thready-new orchestration over those engines. Where an engine is a scaffold/stub (HelixLLM default embedder, VisionEngine OCR, Herald Max, MeTube webhook, Security-KMP mobile storage), the diagram shows the *target* state and the corresponding sibling doc carries the [GAP: ...] and the plan to close it. Nothing here claims a stub “works”.

8. Gap-register coverage

This overview does not itself close gaps; it routes them. Each [GAP: ...] from `helix_thready_subsystem_gaps_and_improvements.md` is addressed in the owning sibling doc: Herald/Max (§5.1) → messenger-ingestion; HelixLLM embedder + VectorDB + Embeddings (§2.1/3.1/2.7) → semantic-search; VisionEngine OCR + `helix_skills` engine (§2.6/4.1) → processing-pipeline; Download Manager + Catalogizer + MeTube/Boba (§6.1–6.5) → asset-and-download; auth + Security-KMP (§7.2/7.3) →

security-model; session_orchestrator claim registry (§2.9) → concurrency-and-idempotency; database partitioning (§3.2) → data-flow & component-catalog.

9. Open items

- `[OPEN: OVERVIEW-1]` Exact HTTP/3 gateway topology (single `http3` process fronting all services vs per-service listeners) is deferred to the deployment pack; tracked as a workable item against `vasic-digital/containers` orchestration.
 - `[OPEN: OVERVIEW-2]` Whether the three environments (dev/sta/prod) share one JetStream cluster with subject prefixes or run isolated clusters is a deployment decision; the architecture supports either (subjects are already namespaced by env in `event-model.md`).
-

Made with love ♥ by Helix Development.